

Explanation of application

A tiny **binary sentiment classifier** (Positive = 1, Negative = 0) that reads a short sentence and predicts its sentiment using an **Embedding** → **SimpleRNN** → **Dense(sigmoid)** network.

Dataset & Labels

```
sentences = [
    "I love this movie", "This film was great",
    "What a fantastic experience", "I really enjoyed it",
    "Absolutely wonderful acting",
    "I hate this movie", "This film was terrible",
    "What a bad experience", "I really disliked it",
    "Absolutely horrible acting"
]
labels = [1,1,1,1,1, 0,0,0,0,0]
```

- 10 short sentences (5 positive, 5 negative).

- Balanced labels → avoids class-imbalance issues for this demo.

Tokenization (Text → Integer Sequences)

```
tokenizer = Tokenizer(num_words=50)    # keep top 50 words (by
frequency)
tokenizer.fit_on_texts(sentences)
X = tokenizer.texts_to_sequences(sentences)
print("Word Index:", tokenizer.word_index)
```

- **Tokenizer** builds a vocabulary from the training texts.
- `num_words=50` means: at model time we will keep only the 50 most frequent tokens (IDs 1..49 effectively; 0 is reserved for padding). Tokens with IDs ≥ 50 are treated as out-of-vocabulary if you later set an `oov_token`.
- `texts_to_sequences` maps each word to its integer ID.

Example: If `tokenizer.word_index` contains `{"i":1, "this":2, "film":3, "was":4, "great":5, ...}`, then

- "I love this movie" → `[1, <id(love)>, 2, <id(movie)>]`.

Padding (Uniform Sequence Length)

```
maxlen = 5
```

```
X = pad_sequences(X, maxlen=maxlen)
```

```
y = np.array(labels)
```

- RNNs expect **equal-length** sequences. `pad_sequences` left-pads with zeros to make each sequence length = maxlen (here 5).
- Shape after padding: `X.shape == (10, 5)` (10 sentences, 5 tokens each). Zeros are **padding token**.

Why 5? It's arbitrary and small for demo. Longer sentences get **truncated** (from the start by default). You can control truncation/padding side with `padding='post', truncating='post'`.

Model Architecture

```
model = Sequential()
```

```
model.add(Embedding(input_dim=50, output_dim=8, input_length=maxlen))
```

```
model.add(SimpleRNN(8, activation="tanh"))
```

```
model.add(Dense(1, activation="sigmoid"))
```

Embedding layer

- **Purpose:** Learn a dense vector for each word ID (distributional semantics).
- **Shapes:**
 - Input: `(batch, timesteps=5)` of integer IDs
 - Output: `(batch, timesteps=5, embed_dim=8)` (each token → 8-D vector)
- **Parameters:** `input_dim * output_dim = 50 * 8 = 400` trainable params.

Important: `input_dim` must be $\geq$ the maximum token index your tokenizer will produce (bounded by `num_words`). With `num_words=50`, `input_dim=50` is consistent.

SimpleRNN layer

- **Purpose:** Read the sequence of embeddings and produce a final hidden state summarizing the sentence.
- **Config:** `SimpleRNN(8, activation="tanh")`
- **Input dim D = 8** (embedding size), **units U = 8**.
- **Output shape:** `(batch, 8)` (last hidden state).
- **Parameters:** $U * (U + D + 1)$
 $\rightarrow 8 * (8 + 8 + 1) = 8 * 17 = 136$.

Why tanh? Classic for vanilla RNNs. For more complex text, prefer **GRU/LSTM** to handle longer dependencies.

Dense output layer

- **Purpose:** Binary classification (positive vs negative).
- **Activation:** `sigmoid` → outputs probability `p`
- **Params:** weights $8 \times 1 = 8$ + bias 1 → **9**.

Total params: 400 (Embedding) + 136 (RNN) + 9 (Dense) = **545**.

Compile & Train

```
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X, y, epochs=30, verbose=0)
```

- **Loss:** `binary_crossentropy` fits a sigmoid output for binary labels.
- **Optimizer:** Adam is a solid default.
- **Metrics:** Accuracy for quick feedback.

With such a tiny dataset and 30 epochs, the model will likely **memorize**.

Inference (Predict on New Sentences)

```
test_sentences = ["I enjoyed this film", "I hated this film"]
test_seq = tokenizer.texts_to_sequences(test_sentences)
test_seq = pad_sequences(test_seq, maxlen=maxlen)
pred = model.predict(test_seq)
for s, p in zip(test_sentences, pred):
    print(f"Sentence: {s} -> Sentiment:", "Positive" if p>0.5 else
          "Negative")
```

- The **same tokenizer** is used at inference to ensure word-to-ID mapping consistency.
- Pad to `maxlen=5`.
- Apply sigmoid scores and threshold at **0.5**:
 - $p > 0.5 \rightarrow$ Positive; else Negative.

End-to-End Shapes

- **Training:**
 - $X: (10, 5) \rightarrow \text{Embedding} \rightarrow (10, 5, 8) \rightarrow \text{SimpleRNN} \rightarrow (10, 8) \rightarrow \text{Dense} \rightarrow (10, 1)$

- **Inference:**

- test_seq: $(2, 5) \rightarrow (2, 5, 8) \rightarrow (2, 8) \rightarrow (2, 1)$